웹 기술 이론 7장

비동기 처리와 HTTP 통신

이벤트 루프, Promise의 구조, fetch와 오류 처리, 데이터 저장소

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

학습 목표

① 이벤트 루프로 비동기 코드의 실행 순서를 예측할 수 있다.

② Promise의 상태와 체이닝을 이해하고 async/await로 바꿔 쓸 수 있다.

③ fetch의 오류 처리에서 무엇이 reject되고 무엇이 안 되는지 구분할 수 있다.

④ HTTP의 구성 요소와 주요 상태 코드를 설명할 수 있다.

⑤ 브라우저 저장소 세 가지의 차이와 쓰임을 구분할 수 있다.

이론 7장

핵심 키워드

이벤트 루프 · 콜 스택 · 태스크 큐 · Promise · async/await · fetch · HTTP · CORS · localStorage · 오류 처리

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

이론 7장 · 1

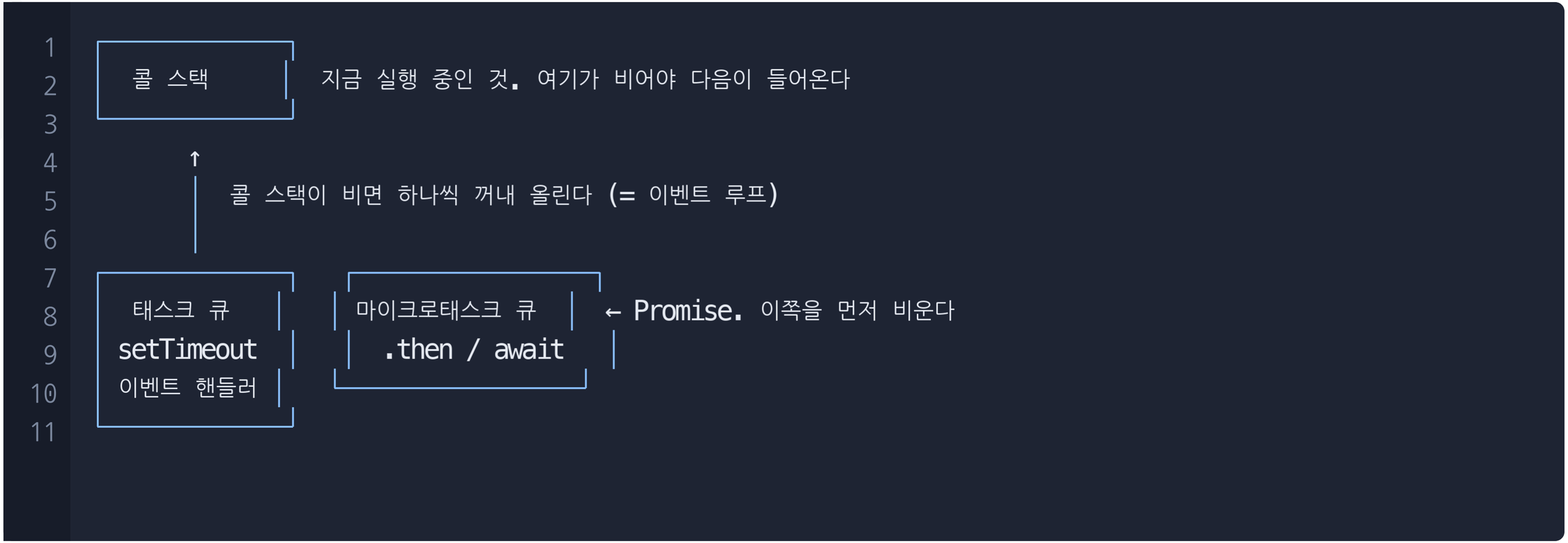
JavaScript는 한 번에 하나만 합니다

JavaScript는 단일 스레드(single-thread) 입니다.

JavaScript는 한 번에 하나만 합니다

1. 한 번에 한 줄씩만 실행합니다.
2. 그런데도 화면이 멈추지 않고 여러 일이 동시에 되는 것처럼 보이는 이유가 이벤트 루프입니다.

이벤트 루프의 구조


콜 스택
지금 실행 중인 것. 여기가 비어야 다음이 들어온다
콜 스택이 비면 하나씩 꺼내 올린다 (= 이벤트 루프)
태스크 큐
setTimeout
이벤트 핸들러
마이크로태스크 큐
.then / await
← Promise. 이쪽을 먼저 비운다

실행 순서 예측하기

```
1  console.log('1');
2
3  setTimeout(() => console.log('2'), 0);        // 태스크 큐
4
5  Promise.resolve().then(() => console.log('3'));  // 마이크로태스크 큐
6
7  console.log('4');
8
9  // 출력: 1 → 4 → 3 → 2
10 // ① 동기 코드(1, 4)를 다 실행
11 // ② 마이크로태스크 큐를 비운다 (3)
12 // ③ 그다음 태스크 큐 (2)
```

JavaScript는 한 번에 하나만 합니다

1. **setTimeout(fn, 0) 은 "지금 당장"이 아니라 "지금 하던 일이 다 끝나면 가장 먼저" 라는 뜻입니다.**

2. **무거운 작업 중간에 화면을 갱신하고 싶을 때 이 성질을 이용하기도 합니다.**

이론 7장 · 2

Promise

Promise

상태	뜻	전이
pending	대기 — 아직 결과 없음	시작 상태
fulfilled	이행 — 값이 들어옴	pending에서 한 번만
rejected	거부 — 오류가 들어옴	pending에서 한 번만

Promise

1. 한 번 결정되면 다시 바뀌지 않습니다.
2. 그래서 .then 을 나중에 붙여도 이미 완료된 결과를 받을 수 있습니다.

체이닝 — 각 then이 새 Promise를 돌려준다

```
fetch(url)
  .then(res => res.json())          // 이 반환값이 다음 then으로
  .then(data => data.items)
  .then(items => render(items))
  .catch(err => showError(err))     // 위 어디서든 실패하면 여기로
  .finally(() => hideSpinner());    // 성공 · 실패 상관없이

/* then 안에서 return을 빠뜨리면 다음 then이 undefined를 받습니다.
   화살표 함수의 중괄호를 쓸 때 특히 자주 하는 실수입니다. */
.then(res => { res.json() })        // ✗ return 없음
.then(res => { return res.json() }) // ✓
.then(res => res.json())            // ✓ 더 간단
```

Promise 조합 메서드

```
// 전부 성공해야 함 — 하나라도 실패하면 즉시 reject
const [a, b] = await Promise.all([fetch(u1), fetch(u2)]);

// 각각의 성공·실패를 모두 받음 — 하나가 실패해도 나머지를 얻는다
const results = await Promise.allSettled([fetch(u1), fetch(u2)]);
results.forEach(r => {
  if (r.status === 'fulfilled') console.log(r.value);
  else console.warn(r.reason);
});

// 가장 먼저 끝난 것 하나 — 타임아웃 구현에 씀
await Promise.race([fetch(url), timeout(5000)]);
```

2.1 여러 개를 한꺼번에

1. 요청 세 개를 순서대로 await 하면 시간이 세 배 걸립니다.
2. 서로 의존하지 않는 요청은 Promise.all 로 동시에 보내세요.
3. 실무에서 체감 속도가 크게 달라집니다.

이론 7장 · 3

async / await

async/await 는 Promise를 동기 코드처럼 위에서 아래로 읽히게 하는 문법입니다.

같은 일, 다른 모양 (1/2)

```
// .then 방식
function load() {
  return fetch(url)
    .then(res => res.json())
    .then(data => data.items)
    .catch(err => { console.error(err); return []; });
}

// async/await 방식
async function load() {
```

> **내부적으로는 여전히 Promise입니다.**

같은 일, 다른 모양 (2/2)

```
  try {
    const res = await fetch(url);
    const data = await res.json();
    return data.items;
  } catch (err) {
    console.error(err);
    return [];
  }
}
```

자주 하는 실수 세 가지

정리

1. **await 를 빠뜨림**
   값 대신 Promise { <pending> } 이 담깁니다. 가장 흔합니다.

2. **try/catch 를 안 씀**
   통신은 실패하는 것이 정상입니다. 처리하지 않으면 unhandled rejection이 됩니다.

3. **순서대로 안 해도 되는 것을 순서대로 await**
   시간이 배로 걸립니다. Promise.all 을 쓰세요.

async 함수는 항상 Promise를 돌려줍니다

```
async function f() { return 1; }

f();            // Promise { 1 }  ← 1이 아닙니다
await f();      // 1
f().then(v => console.log(v));   // 1

/* 그래서 async 함수를 부른 쪽도 결과를 쓰려면 await하거나 then을 붙여야 합니다. */
```

이론 7장 · 4

HTTP

실제 모양

```
── 요청 ──────────────────────────────────
GET /api/students/3 HTTP/1.1          시작줄: 메서드 · 경로 · 버전
Host: example.com                      헤더
Accept: application/json
Authorization: Bearer eyJhbGc...
                                       빈 줄
(본문 — GET에는 보통 없음)

── 응답 ──────────────────────────────────
HTTP/1.1 200 OK                       상태줄: 버전 · 코드 · 설명
Content-Type: application/json         헤더
Cache-Control: max-age=3600

{ "id": 3, "name": "김한신" }          본문
```

4.2 메서드와 성질

메서드	용도	안전한가	멱등인가
GET	조회	예 (바꾸지 않음)	예
POST	생성	아니오	아니오 (두 번 보내면 두 개 생김)
PUT	통째로 교체	아니오	예
PATCH	일부 수정	아니오	경우에 따라
DELETE	삭제	아니오	예

4.2 메서드와 성질 (1/2)

1. 멱등(idempotent) 은 "몇 번 보내도 결과가 같다"는 뜻입니다.
2. 네트워크가 불안해 재시도할 때 안전한지를 판단하는 기준이 됩니다.
3. POST가 멱등이 아니라서 "결제 버튼을 두 번 눌러 두 번 결제되는" 사고가 납니다.

4.2 메서드와 성질 (2/2)

1. GET으로 자료를 바꾸면 안 됩니다.
2. GET은 안전하다고 약속된 메서드라 브라우저가 미리 불러오거나 캐시할 수 있고, 검색엔진 크롤러도 자유롭게 방문합니다.
3. /delete?id=3 같은 GET 주소를 만들었다가 크롤러가 지나가며 자료를 전부 지운 사고가 실제로 있었습니다.

4.3 상태 코드 (1/2)

코드	뜻	흔한 상황
200 OK	성공	정상 조회
201 Created	생성됨	POST 성공
204 No Content	성공, 본문 없음	DELETE 성공
301 / 302	영구 / 임시 이동	주소 변경
304 Not Modified	안 바뀜	캐시 사용
400 Bad Request	요청이 잘못됨	보낸 값이 형식에 안 맞음

4.3 상태 코드 (2/2)

코드	뜻	흔한 상황
401 Unauthorized	인증 필요	로그인 안 됨
403 Forbidden	권한 없음	로그인은 했지만 권한 부족
404 Not Found	없음	주소 오타, 파일 없음
409 Conflict	충돌	이미 있는 이메일로 가입
500 / 502 / 503	서버 오류	서버 쪽 문제

이론 7장 · 5

fetch의 오류 처리

무엇이 reject되고 무엇이 안 되는가

```
try {
  const res = await fetch(url);

  // 여기까지 왔다는 것은 "서버가 응답을 주긴 줬다"는 뜻입니다.
  //    404도 500도 여기까지 옵니다. reject되지 않습니다.
  if (!res.ok) {                    // res.ok === (200 <= status < 300)
    throw new Error(`HTTP ${res.status}`);
  }

  const data = await res.json();    // 본문이 JSON이 아니면 여기서 던짐
  return data;

} catch (e) {
  // reject되는 경우: 네트워크 끊김, DNS 실패, CORS 차단, 요청 취소
  // + 위에서 우리가 직접 throw한 것
  console.error(e.message);
}
```

fetch를 쓸 때의 체크리스트

정리

1. **res.ok 를 확인했는가**
   안 하면 404 응답의 오류 페이지를 정상 데이터로 착각합니다.
2. **try/catch 로 감쌌는가**
   네트워크는 끊깁니다.
3. **로딩 상태를 화면에 표시했는가**
   없으면 사용자에게 먹통으로 보입니다.
4. **실패 상태를 화면에 표시했는가**
   콘솔에만 찍으면 사용자는 아무것도 모릅니다.

5.1 CORS

1. 브라우저는 다른 출처(origin)로 보내는 요청을 기본적으로 막습니다.

2. 출처는 프로토콜 + 호스트 + 포트 세 가지가 모두 같아야 같은 것으로 봅니다.

이런 오류를 만나면

```
Access to fetch at 'https://api.example.com/data'
from origin 'http://localhost:5500' has been blocked by CORS policy:
No 'Access-Control-Allow-Origin' header is present.

/* 이건 내 코드의 문법 오류가 아닙니다.
   서버가 "이 출처는 허용한다"는 응답 헤더를 보내야 통과됩니다.
   프론트엔드에서 할 수 있는 것이 없습니다.

   → 해결: CORS를 허용한 공개 API를 쓰거나, 서버를 직접 만들거나,
           개발 중에는 프록시를 씁니다. */
```

이론 7장 · 6

브라우저 저장소

브라우저 저장소

	localStorage	sessionStorage	쿠키
수명	지울 때까지	탭을 닫으면 사라짐	만료일까지
용량	5~10MB	5~10MB	4KB
서버 전송	안 됨	안 됨	요청마다 자동 전송
JS 접근	가능	가능	HttpOnly면 불가
쓰임	설정·임시 저장	한 세션 동안만	로그인 세션

안전하게 쓰는 형태 (1/2)

```
const KEY = 'app-todos';

function save(value) {
  try {
    localStorage.setItem(KEY, JSON.stringify(value));
  } catch (e) {
    // 시크릿 모드, 용량 초과, 브라우저 설정에서 차단 — 실제로 던집니다
    console.warn('저장 실패', e);
  }
}
```

안전하게 쓰는 형태 (2/2)

```
function load(fallback = []) {
  try {
    const raw = localStorage.getItem(KEY);
    return raw ? JSON.parse(raw) : fallback;
  } catch {
    return fallback;   // 저장된 값이 깨졌을 수도 있습니다
  }
}
```

localStorage에 넣으면 안 되는 것

정리

1. **비밀번호·주민번호·카드번호**
   JavaScript로 누구나 읽을 수 있고, XSS가 통하면 그대로 털립니다.
2. **로그인 토큰**
   논쟁이 있지만, 보안이 중요한 서비스는 HttpOnly 쿠키를 씁니다.
3. **사라지면 안 되는 자료**
   사용자가 브라우저를 정리하면 없어집니다. 서버에 두어야 할 것은 서버에.
4. **다른 기기에서도 봐야 하는 것**
   그 브라우저에만 있습니다.

브라우저 저장소

1. 학습·연습 단계에서는 localStorage로 충분합니다.
2. "이건 이 브라우저에만 남는다" 는 것만 정확히 알고 쓰면 됩니다.

웹 기술 이론 7장 마무리

7장 마무리

비동기 처리와 HTTP 통신 — 본문과 도해는 학습사이트에서 다시 읽습니다

- ✔ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시
- ✔ 함께 볼 주차: 12주차 · 13주차
- ✔ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다